

Gradient Descent and Block Coordinate Descent for Multiclass Logistic Regression

Gioele D’Osvaldi

Riccardo Pilan

Federico Russo

Abstract

This report presents the implementation and analysis of three first-order optimization algorithms on a multiclass logistic regression task: Gradient Descent, Randomized Block Coordinate Gradient Descent and Gauss-Southwell Block Coordinate Gradient Descent. The problem to be addressed is presented through its mathematical formulation, followed by the application of the algorithms. These are first tested on a synthetic dataset and then analyzed for performance with respect to a real-world dataset aimed at recognizing human activities from smartphone sensors.

1 Introduction

This report explores the implementation and the comparative analysis of multiclass logistic regression, focusing on three optimization algorithms: standard Gradient Descent (GD), Block Coordinate Descent using a randomized selection rule (Randomized BCGD), and Block Coordinate Descent employing the greedy Gauss-Southwell rule (Gauss-Southwell BCGD).

The presentation begins with the mathematical formulation of the problem, highlighting the techniques employed to achieve greater computational efficiency, followed by a detailed explanation of the implementation of the algorithms.

To ensure numerical stability and verify algorithmic correctness, an experimental phase is conducted by testing the algorithms on a high-dimensional synthetic dataset, consisting of a randomly generated 1000×1000 feature matrix and considering 50 target classes.

Subsequently, the optimization methods are applied to a real-world Human Activity Recognition dataset, to assess their practical viability and generalization capabilities.

The report concludes with a comparative analysis of the three algorithms, focusing specifically on the trade-off between classification accuracy and computational efficiency, measured in terms of CPU time.

2 Formulation

The optimization problem that needs to be solved consists of the minimization of the objective function of a multiclass logistic problem of the form:

$$\min_{X \in \mathbb{R}^{d \times k}} \sum_{i=1}^m \left[-x_{b_i}^\top a_i + \log \left(\sum_{c=1}^k \exp(x_c^\top a_i) \right) \right] \quad (1)$$

Let $a_i \in \mathbb{R}^d$ denote the i -th feature vector of the data matrix $A \in \mathbb{R}^{m \times d}$, defined as a column vector, corresponding to the i -th training sample. Therefore:

$$A = \begin{bmatrix} a_1^\top \\ a_2^\top \\ \vdots \\ a_m^\top \end{bmatrix}.$$

Given a training set of m identically distributed training samples, where each sample i has a feature vector $a_i \in \mathbb{R}^d$ and a class $b_i \in \{1, 2, \dots, k\}$, the model likelihood is:

$$p(b_i | a_i, X) = \frac{\exp(x_{b_i}^\top a_i)}{\sum_{c=1}^k \exp(x_c^\top a_i)} \quad (2)$$

where x_c denotes the c -th column of the matrix X .

Hence, the objective is to minimize the negative log-likelihood:

$$f(X) = \sum_{i=1}^m \left[-x_{b_i}^\top a_i + \log \left(\sum_{c=1}^k \exp(x_c^\top a_i) \right) \right] \quad (3)$$

For first-order optimization, the partial derivative with respect to entry X_{jc} is defined as:

$$\frac{\partial f(X)}{\partial X_{jc}} = - \sum_{i=1}^m a_{ij} \left[\mathbf{1}(b_i = c) - \frac{\exp(x_c^\top a_i)}{\sum_{c'=1}^k \exp(x_{c'}^\top a_i)} \right] \quad (4)$$

To implement the loss function ($f(X)$) and its gradient ($\nabla f(X)$), the previous definitions have been rewritten in matrix form (vectorization), so as to operate simultaneously on the entire data matrix $A \in \mathbb{R}^{m \times d}$, on the weight matrix $X \in \mathbb{R}^{d \times k}$, and on the label matrix in one-hot encoding format $I \in \mathbb{R}^{m \times k}$ to increase computational efficiency.

Defining the matrix product $Z = AX \in \mathbb{R}^{m \times k}$, the following is obtained:

$$f(X) = -\langle I, Z \rangle_F + \sum_{i=1}^m \log \left(\sum_{c=1}^k \exp(Z_{ic}) \right) \quad (5)$$

To understand the term $\langle I, Z \rangle_F$, the scalar term $-x_{b_i}^\top a_i$ must be considered. This term evaluates the score calculated for the i -th sample corresponding strictly to the correct class b_i . In the matrix formulation, the matrix I acts as a “mask” containing a 1 only at the position corresponding to the true class for each sample. Consequently, computing the Frobenius inner product $\langle I, Z \rangle_F = \text{Tr}(I^\top Z)$ extracts and sums only the logits associated with the correct classes, discarding the rest. From an implementation standpoint, this is computed as an element-wise multiplication between I and Z , followed by a global sum.

Finally, defining the probability matrix $P = \text{softmax}(Z) \in \mathbb{R}^{m \times k}$, Equation (4) is rewritten in matrix form:

$$\nabla f(X) = A^\top (P - I) \quad (6)$$

3 Algorithms

3.1 Implementation of the Loss function and the Gradient

Before analyzing the structure of the optimization methods, the implementation of the function to be minimized and its gradient is presented. Based on the vectorized form derived in Equations 5 and 6, for-loops used to compute the functions were replaced by vectorized operations using the *Numpy* library, significantly reducing execution time. Furthermore, to prevent overflow errors when evaluating the exponentials of large logits, the second term of the objective Function 5 was implemented using the LogSumExp trick provided by the SciPy library.

3.2 Choice of the Algorithms

To solve the multiclass logistic regression problem, three first-order methods are considered, taking advantage of the differentiability and convexity of the objective function: Gradient Descent (GD), Block Coordinate Descent with a Randomized rule (BCGD Randomized), and Block Coordinate Descent with the Gauss–Southwell rule (BCGD Gauss–Southwell).

3.3 Gradient Descent

Gradient Descent (GD) is a fundamental first-order optimization algorithm used to minimize the objective function. Starting from an initial weight matrix, the algorithm moves towards the minimum by iteratively updating the parameters in the direction of the steepest descent.

The implementation of the GD algorithm is summarized in pseudocode 1.

Algorithm 1 Gradient Descent (GD)

Require: Data matrix $A \in \mathbb{R}^{m \times d}$, label indicator matrix $I \in \mathbb{R}^{m \times k}$, maximum number of iterations n_{iter} , tolerance $\varepsilon > 0$, stepsize multiplier h , boolean *use_armijo*

```
1: Initialize weight matrix  $X^{(0)} \in \mathbb{R}^{d \times k} \sim \mathcal{N}(0, 0.01^2)$ 
2: Compute initial score matrix  $Z^{(0)} \leftarrow AX^{(0)}$ 
3: Compute Lipschitz constant:  $L \leftarrow \|A\|_2^2$ 
4: for  $t = 0, 1, \dots, n_{iter} - 1$  do
5:   Compute full gradient  $\nabla f(X^{(t)})$ 
6:   if  $\|\nabla f(X^{(t)})\|_F < \varepsilon$  then ▷ Convergence check
7:      $X_{final} \leftarrow X^{(t)}$ 
8:     break
9:   end if
10:  if use_armijo then
11:    Determine stepsize  $\alpha^{(t)}$  using Armijo rule
12:  else
13:     $\alpha^{(t)} \leftarrow h/L$  ▷ Constant stepsize with multiplier
14:  end if
15:   $X^{(t+1)} \leftarrow X^{(t)} - \alpha^{(t)} \nabla f(X^{(t)})$ 
16:  Update matrix product  $Z^{(t+1)} \leftarrow AX^{(t+1)}$ 
17: end for
18:  $X_{final} \leftarrow X^{(t+1)}$ 
19: return  $X_{final}$ 
```

3.4 Randomized Block Coordinate Gradient Descent

The Randomized Block Coordinate Gradient Descent algorithm (Randomized BCGD) is a variation of Gradient Descent (GD) in which, at each iteration t , a specific block of coordinates is selected at random, and only the corresponding row of the weight matrix X is updated, thus requiring the computation of only a portion of the full gradient. Updating a single block, rather than the whole matrix, is significantly cheaper and less memory-intensive than evaluating the full gradient at every iteration. At each iteration, a feature index j is selected uniformly at random with probability $1/d$. The partial gradient is then evaluated and used to update the variables in order to minimize the objective function. This makes Randomized BCGD the only algorithm among those considered in this report in which the full gradient is never evaluated to update the weight matrix. However, the full gradient is still required to assess convergence of the algorithm, as discussed in Section 3.8

The algorithm is defined in pseudocode as follows:

Algorithm 2 Randomized Block Coordinate Gradient Descent

Require: Data matrix $A \in \mathbb{R}^{m \times d}$, label indicator matrix $I \in \mathbb{R}^{m \times k}$, max iterations n_{iter} , tolerance ε , stepsize multiplier h , check frequency e

Ensure: Final weight matrix X_{final}

- 1: Initialize weight matrix $X^{(0)} \in \mathbb{R}^{d \times k} \sim \mathcal{N}(0, 0.01^2)$
- 2: Compute initial score matrix $Z^{(0)} \leftarrow AX^{(0)}$
- 3: Compute partial Lipschitz constants $L_j \leftarrow \|A_{:,j}\|_2^2 \quad \forall j \in \{1, \dots, d\}$
- 4: **for** $t = 0, 1, \dots, n_{iter} - 1$ **do**
- 5: **if** $t \pmod{e} = 0$ **then**
- 6: Compute full gradient $\nabla f(X^{(t)})$
- 7: **if** $\|\nabla f(X^{(t)})\|_F < \varepsilon$ **then** ▷ Convergence check
- 8: $X_{final} \leftarrow X^{(t)}$
- 9: **break**
- 10: **end if**
- 11: **end if**
- 12: A random index $j \in \{1, \dots, d\}$ is selected with uniform probability $1/d$
- 13: **if** h is specified **then**
- 14: $\alpha_j \leftarrow h/L_j$
- 15: **else**
- 16: $\alpha_j \leftarrow 1/L_j$
- 17: **end if**
- 18: Compute class probabilities: $P \leftarrow \text{softmax}(Z^{(t)})$
- 19: Compute partial gradient for block j : $\nabla_j f(X^{(t)}) \leftarrow A_{:,j}^\top (P - I)$
- 20: Compute weight change: $\Delta X_{j,:} \leftarrow -\alpha_j \nabla_j f(X^{(t)})$
- 21: Update weight matrix: $X_{j,:}^{(t+1)} \leftarrow X_{j,:}^{(t)} + \Delta X_{j,:}$
- 22: Update score matrix incrementally: $Z^{(t+1)} \leftarrow Z^{(t)} + A_{:,j} \Delta X_{j,:}$ ▷ Rank-1 outer product
- 23: **end for**
- 24: $X_{final} \leftarrow X^{(t+1)}$
- 25: **return** X_{final}

While this approach is faster in terms of time per iteration, updating the weights based on a single partial gradient requires a higher number of iterations to reach convergence. Therefore, this method may be slower than standard GD in terms of total execution time.

Moreover, although only one coordinate block is updated, probabilities are still evaluated for all observations, since the softmax depends on the full score matrix $Z^{(t)}$.

Unlike deterministic methods, the convergence of Randomized BCGD is characterized in terms of the expected value of the objective function, $\mathbb{E}[f(X_t)]$. Under standard assumptions, this expected value converges to the optimal solution $f(X^*)$.

3.5 Gauss-Southwell Block Coordinate Gradient Descent

While standard Gradient Descent updates all parameters simultaneously, Block Coordinate Descent (BCGD) methods (both with a Randomized rule and the Gauss-Southwell rule) update a single subset of variables (specifically, a row of the weight matrix) at each iteration. Unlike the Randomized variant, which is characterized by a uniform random choice of the row, the Gauss-Southwell (GS) rule employs a greedy approach: it evaluates the full gradient and iteratively selects the coordinate block with the maximum gradient norm, which corresponds to the direction of steepest descent.

The implementation of the GS-BCGD algorithm is summarized in Pseudocode [3](#)

Algorithm 3 Gauss-Southwell BCGD

Require: Data matrix $A \in \mathbb{R}^{m \times d}$, label indicator matrix $I \in \mathbb{R}^{m \times k}$, maximum number of iterations n_{iter} , tolerance ε , stepsize multiplier h

Ensure: Final weight matrix X_{final}

```

1: Initialize weight matrix  $X^{(0)} \in \mathbb{R}^{d \times k} \sim \mathcal{N}(0, 0.01^2)$ 
2: Compute initial score matrix  $Z^{(0)} \leftarrow AX^{(0)}$ 
3: Compute partial Lipschitz constants  $L_j \leftarrow \|A_{:,j}\|_2^2 \quad \forall j \in \{1, \dots, d\}$ 
4: for  $t = 0, 1, \dots, n_{iter} - 1$  do
5:   Compute full gradient  $\nabla f(X^{(t)})$ 
6:   if  $\|\nabla f(X^{(t)})\|_F < \varepsilon$  then ▷ Convergence check
7:      $X_{final} \leftarrow X^{(t)}$ 
8:     break
9:   end if
10:  Compute row norms:  $n_i \leftarrow \|\nabla_i f(X^{(t)})\|_2 \quad \forall i \in \{1, \dots, d\}$ 
11:  Select feature index with largest norm:  $j \leftarrow \arg \max_{i \in \{1, \dots, d\}} (n_i)$ 
12:  if  $h$  is specified then
13:     $\alpha_j \leftarrow h/L_j$ 
14:  else
15:     $\alpha_j \leftarrow 1/L_j$ 
16:  end if
17:  Compute weight change for block  $j$ :  $\Delta X_{j,:} \leftarrow -\alpha_j \nabla_j f(X^{(t)})$ 
18:  Update weight matrix:  $X_{j,:}^{(t+1)} \leftarrow X_{j,:}^{(t)} + \Delta X_{j,:}$ 
19:  Update score matrix incrementally:  $Z^{(t+1)} \leftarrow Z^{(t)} + A_{:,j} \Delta X_{j,:}$  ▷ Rank-1 outer product
20: end for
21:  $X_{final} \leftarrow X^{(t+1)}$ 
22: return  $X_{final}$ 

```

3.6 Initialization

The weight matrix $X^{(0)}$ is initialized by sampling its entries from a standard normal distribution $\mathcal{N}(0, 1)$ scaled by a factor of 0.01. This approach was selected to promote numerical stability during the computation of the softmax probabilities, while simultaneously breaking the symmetry of the initial weights. In addition, a fixed random seed (42) is set to ensure the full reproducibility of the experimental results.

3.7 Stepsize selection

The three algorithms implement different methods for calculating the stepsize:

Gradient Descent:

- **Fixed stepsize:** The stepsize is defined using the Lipschitz constant of the gradient, L , as $\alpha = 1/L$. Mathematically, with this choice of stepsize, the objective function is guaranteed to decrease monotonically at every iteration without any risk of divergence.

To compute L for the multiclass logistic regression objective, the maximum eigenvalue of the Hessian matrix must be evaluated. From the gradient formulation in Equation (6), the Hessian is derived as:

$$\nabla^2 f(X) = \sum_{i=1}^m H_i \otimes (a_i a_i^\top) \quad (7)$$

where $\otimes$ denotes the Kronecker product and H_i is the covariance matrix of the probabilities for the i -th sample. Based on the properties of the multinomial distribution, since the probabilities lie in the interval $[0, 1]$ and sum to 1, the eigenvalues of H_i are bounded above by 1. This implies the matrix inequality:

$$H_i \leq I_k \quad (8)$$

where I_k is the $k \times k$ identity matrix. Substituting this inequality, the Hessian is bounded by:

$$\nabla^2 f(X) \leq \sum_{i=1}^m I_k \otimes (a_i a_i^\top) = I_k \otimes (A^\top A) \quad (9)$$

by the property of linearity of the Kronecker product.

Since the Lipschitz constant is given by the maximum eigenvalue of the Hessian, Equation (9) demonstrates that this value is bounded by the maximum eigenvalue of $A^\top A$, which corresponds to the squared spectral norm $\|A\|_2^2$. Therefore, the stepsize implemented in GD is $\alpha = 1/\|A\|_2^2$.

- **Armijo Rule:** The stepsize $\alpha^{(t)}$ is dynamically calculated through a backtracking line search. This ensures a sufficient decrease in the objective function without causing divergence. The Armijo rule is presented in Pseudocode 4.

Given a descent direction $D^{(t)} = -\nabla f(X^{(t)})$, starting from an initial guess $\Delta^{(t)}$, the stepsize is iteratively reduced by a factor $\delta \in (0, 1)$ until the following inequality is satisfied:

$$f(X^{(t)} + \alpha^{(t)} D^{(t)}) \leq f(X^{(t)}) + \gamma \alpha^{(t)} \langle \nabla f(X^{(t)}), D^{(t)} \rangle_F = f(X^{(t)}) - \gamma \alpha^{(t)} \|\nabla f(X^{(t)})\|_F^2 \quad (10)$$

This method makes the GD algorithm robust to variations in the curvature, though it requires multiple evaluations of the objective function per iteration, making it computationally expensive.

Algorithm 4 Armijo Rule

Require: Current point $X^{(t)}$, descent direction $D^{(t)}$, initial stepsize $\Delta^{(t)} > 0$, fixed parameters $\delta \in (0, 1)$ and $\gamma \in (0, 1/2)$

- 1: $\alpha \leftarrow \Delta^{(t)}$
- 2: **while** $f(X^{(t)} + \alpha D^{(t)}) > f(X^{(t)}) + \gamma \alpha \langle \nabla f(X^{(t)}), D^{(t)} \rangle_F$ **do**
- 3: $\alpha \leftarrow \delta \alpha$
- 4: **end while**
- 5: **return** α

Block Coordinate Descent:

While standard GD employs the global Lipschitz constant $L = \|A\|_2^2$, both Block Coordinate Descent (BCGD) methods (Randomized and Gauss-Southwell) require partial Lipschitz constants L_j reflecting the local curvature along the specific j -th feature.

The local Hessian for block j can be derived from Equation (7) and expressed as the sum of the covariance matrices H_i , weighted by the squared values of the j -th feature:

$$\nabla_{X_j}^2 f(X) = \sum_{i=1}^m A_{i,j}^2 (\text{diag}(P_{i,:}) - P_{i,:}^\top P_{i,:}) = \sum_{i=1}^m A_{i,j}^2 H_i \quad (11)$$

Applying the same bound ($H_i \leq I_k$) from Equation (8), the local Hessian is bounded by:

$$\nabla_{X_j}^2 f(X) \leq \left(\sum_{i=1}^m A_{i,j}^2 \right) I_k \quad (12)$$

Therefore, since the partial Lipschitz constant L_j is the maximum eigenvalue of the local Hessian, it corresponds to the squared Euclidean norm of the j -th feature column $A_{:,j}$:

$$L_j = \|A_{:,j}\|_2^2 = \sum_{i=1}^m A_{i,j}^2 \quad (13)$$

The array of Lipschitz partial constants $\{L_j\}_{j=1}^d$ is computed before the main loop to ensure computational efficiency. Theoretically, the stepsizes α_j used as a baseline for the application of the algorithms are the inverse of the partial Lipschitz constants ($1/L_j$). However, to allow for hyperparameter tuning and potentially reduce the number of iterations to accelerate convergence, a scaling parameter h is introduced, leading to the effective stepsize selection of $\alpha = h/L_j$.

3.8 Stopping criteria

The primary stopping criterion is based on the Frobenius norm of the full gradient of the objective function:

$$\|\nabla f(X^{(t)})\|_F < \varepsilon \quad (14)$$

where $\varepsilon > 0$ is a predefined tolerance threshold.

This norm represents a measure of the distance from a stationary point, and in convex optimization problems, a small gradient indicates proximity to the global minimum. Since both standard GD and BCGD with the Gauss-Southwell rule require the computation of the full gradient at each step, evaluating the stopping condition at each iteration is not computationally expensive. Conversely, in the context of the Randomized BCGD algorithm, since updates only require a partial gradient calculation, the stopping condition is evaluated periodically, every e iterations, to limit the computational cost of the additional gradient evaluation.

The tolerance value must be selected considering the scaling of the objective function, which is defined in Equation (3).

The parameter e introduces a fundamental trade-off between the accuracy of the criterion and the computational efficiency by controlling the frequency of the stopping check. Smaller values of e lead to more frequent evaluations and potentially higher accuracy in detecting convergence, at the expense of a higher computational cost, while larger values of e reduce the computational toll, but may delay convergence detection and lead to unnecessary iterations.

A secondary stopping criterion is the maximum number of iterations, forcing the interruption of the algorithm: this condition can be triggered even without reaching full convergence.

4 Dataset

Two main datasets were utilized for the experimental evaluation. The first is a synthetic dataset, generated according to the project specifications, which was primarily employed to verify the algorithmic correctness of the implementations. To further evaluate the performance of the optimization methods on a real-world scenario, the Human Activity Recognition (HAR) using Smartphones dataset [1] was selected.

The objective of the HAR dataset is to classify human physical activities based on data collected from smartphone sensors. The data collection involved 30 volunteers, aged between 19 and 48. During the experiment, each subject wore a smartphone on their waist, and signals from the device’s accelerometer and gyroscope were recorded.

The raw data were then processed using feature engineering techniques to extract a vector of $d = 561$ features. Each participant performed multiple activities, recorded in 2.56-second time windows, leading to a total dataset size of $m = 10,299$ instances. In particular, $k = 6$ distinct activity classes were labelled: Walking, Walking Upstairs, Walking Downstairs, Sitting, Standing, and Laying.

5 Results

5.1 Experimental Setup

Having presented the algorithms from a theoretical perspective and detailed the implementation, this section analyzes the results obtained by applying these methods to the dataset introduced in section 4. To ensure a fair comparison, all algorithms are tested within the same experimental setup with fixed random seed of 42, utilizing the same training and testing datasets and stopping criteria. Furthermore, all testing was performed on the same machine to ensure reliable CPU time measurement.

Metrics of Interest: Performance is evaluated through two complementary perspectives:

- a) **Algorithmic efficiency:** expressed as the number of iterations required to reach convergence, measuring how fast the algorithm progresses from a numerical standpoint.
- b) **Computational efficiency:** measured as the CPU time required to execute the algorithms, reflecting the actual execution cost.

An algorithm may be fast in terms of iterations but slow in terms of CPU time per iteration; therefore, considering both perspectives provides a more detailed understanding of its performance. Additionally, predictive accuracy is evaluated using the following metric:

$$\text{Accuracy} = \frac{1}{m_{\text{test}}} \sum_{i=1}^{m_{\text{test}}} \mathbb{I}(\text{argmax}(Z_{i,:}) = b_i) \quad (15)$$

Results on synthetic dataset: Before analyzing the performance on real-world data, the methods were tested on the generated 1000x1000 feature matrix with the purpose described in Section 4. Since the primary goal of this part of the testing is to run a preliminary check on the implemented methods and not to assess the overall accuracy or CPU time, the training and the testing phases were conducted on the same dataset A. It was observed that every algorithm reached convergence with an accuracy close to 100%. The fastest method was the standard GD that converged in just 1 iteration (0.04 seconds) while the other two algorithms took between 10 and 20 times longer (depending on the parameters). This behavior was also observed on the publicly available dataset, which will be described in the following sections.

Tolerance parameter: To ensure a fair comparison, all algorithms employ the same stopping criterion, as discussed in Section 3.8. While the following analysis illustrates how accuracy and iterations vary across tolerance levels, the main comparison is conducted using a fixed tolerance of $\varepsilon = 500$. In dataset 1, the objective function and the full gradient are computed as a sum over the $m = 7352$ training samples rather than an average, so the stopping criterion scales with the dataset size. Consequently, a total gradient norm of 500 corresponds to a small per-sample variation, with an average gradient norm of approximately 0.068.

5.2 Convergence Speed and Accuracy

Different stepsizes are selected due to the algorithms' nature: $50/L$ for GD, $5/L_j$ for Randomized BCGD, and $9/L_j$ for Gauss-Southwell.

- **CPU time:** Before comparing the three algorithms, the two stepsize strategies tested for GD are evaluated: the Armijo rule and a fixed stepsize. Looking at Figure 1, for a fixed value of 1000 iterations, the implementation of the Armijo rule causes a drastic slowdown of the algorithm, due to the fact that this implementation requires evaluating the function multiple times to guarantee a stable decrease.

Furthermore, when considering a reference tolerance of 500 for comparing the two algorithms, it can be seen that GD with a fixed stepsize achieves similar accuracy (94.06% vs 94.3%), but with drastically lower CPU time (0.20s vs 3.84s), making

Armijo computationally inefficient. Given these considerations, the Armijo rule was discarded for subsequent analyses.

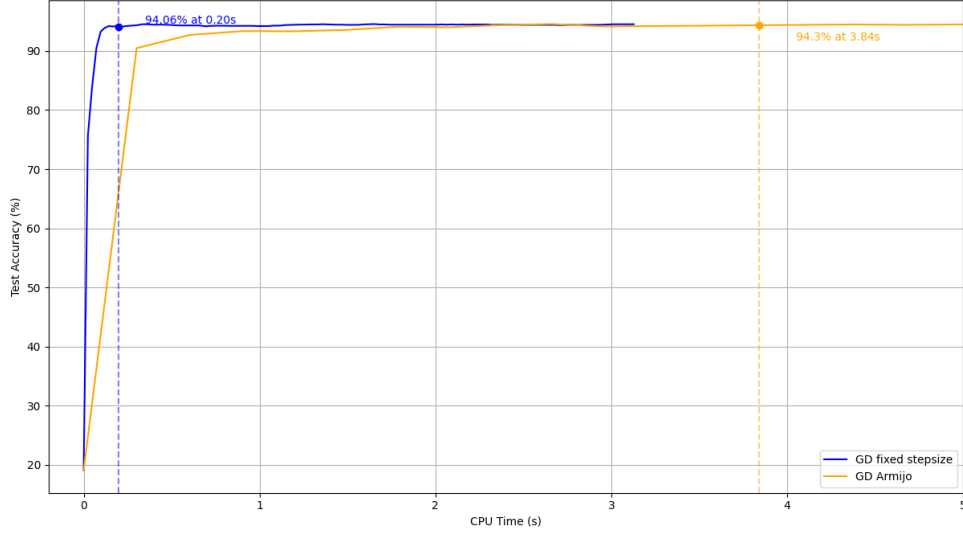


Figure 1: Gradient Descent: Fixed Stepsize vs Armijo Rule

Therefore, a comparison of the three final algorithms is presented below.

Analyzing the graph (Figure 2), a key initial observation is that, in this specific case, the curve with the highest initial rate of change is that of the Gauss-Southwell BCGD. In the first fractions of a second, the green line is the steepest one, exceeding 90% accuracy before the others. This confirms its expected behavior: updating the coordinate with the largest gradient leads to the fastest increase the initial steps. The Randomized BCGD follows: thanks to the random selection of coordinates it manages to pick blocks that often make a significant contribution to the gradient with a minimal per-iteration CPU cost. Thus, considering the first milliseconds, it appears that the BCGD methods are reaching convergence faster. However, at around 0.05 seconds, GD overtakes both BCGD methods, reaching an accuracy of around 94 – 95%. This behavior is explained by the relatively small number of features in the dataset ($d = 561$), which makes full-gradient computations substantially inexpensive for GD. In contrast, block methods require many more iterations, leading to a higher overall computational cost for this problem. Moreover, the speed of GD in achieving this level of accuracy with respect to the other two methods, is also due to:

- The function to be minimized (Equation 3) does not have a mathematical structure that can be exploited in such a way that the implementations of Randomized BCGD and Gauss-Southwell BCGD would be more efficient, due to the presence of Equation 2).
- The chosen real-world dataset (Section 4) is dense. Since BCGD methods update only one block at a time per iteration, a sparse data matrix would significantly reduce the number of multiplications required for the update.

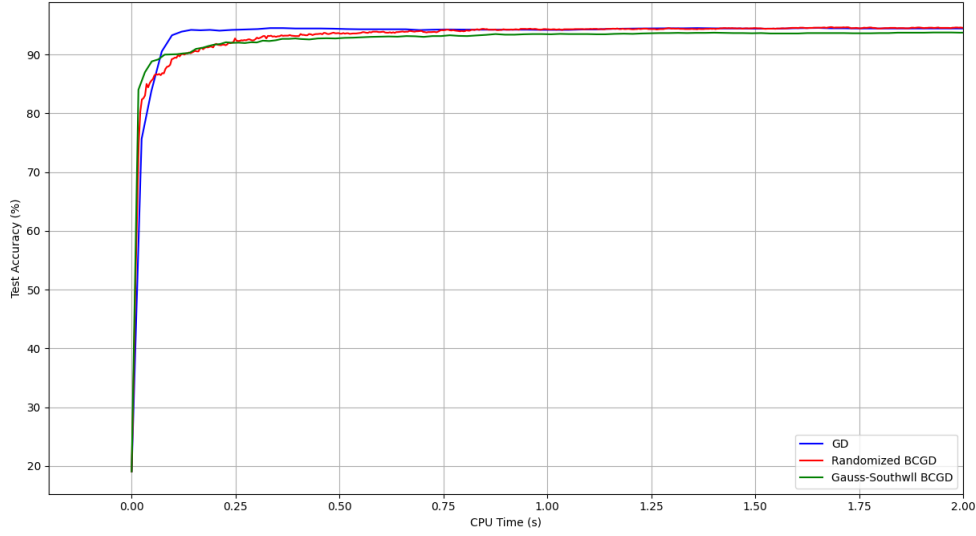


Figure 2: Accuracy vs CPU Time

Hence, the Gauss-Southwell BCGD is designed to select the “best” coordinate, and this choice is highly effective in significantly increasing accuracy during the initial phase. However, the presence of Equation 2 means that the entire gradient must be reevaluated at each iteration due to the presence of the sum of exponentials in the denominator and the need to evaluate the full gradient to apply the arg max in the Gauss-Southwell rule. When compared to the Randomized BCGD, the Gauss-Southwell BCGD quickly loses its initial advantage; in fact, after the vertical ascent phase, the green curve flattens out, while the fixed-step GD continues its ascent, surpassing both the others.

After 1.5 – 2 seconds, a stabilization phase begins in which all three algorithms tend to flatten.

When the tolerance is reduced ($\varepsilon = 100$), Randomized BCGD achieves the best predictive accuracy (94.84%) in 4.36 seconds. Moreover, increasing the stepsize significantly reduces the CPU time, bringing it closer to GD; for example, Gauss-Southwell converges in 0.20s with 91.28% accuracy.

- **Iterations:** GD is the fastest of the three, requiring only 90 iterations to reach convergence with an accuracy of 94.06%. The Gauss-Southwell approach is the best among the block methods, requiring 274 iterations compared to the 1500 of Randomized BCGD. In terms of accuracy, GS BCGD achieves 92.67%, while Randomized BCGD performs slightly better with 93.89%.

The difference in the number of iterations is mainly due to the fact that GD updates all features at each step, while block methods operate on a single coordinate. The Gauss-Southwell method, thanks to its greedy strategy, performs the most informative updates, which explains why it requires about 5 times fewer iterations than Randomized BCGD, where already-updated features may be selected again.

By lowering the tolerance to 100, the same trend is observed. GD requires 515 iterations, GS 2537, and Randomized BCGD 11,500 iterations to reach the desired precision.

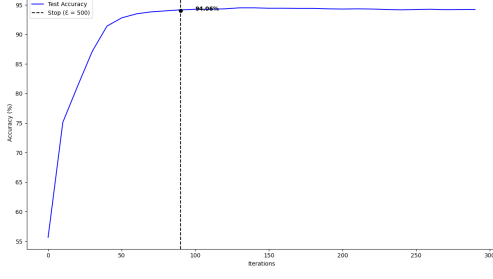


Figure 3: Accuracy vs Iterations ($\varepsilon = 500$) (GD)

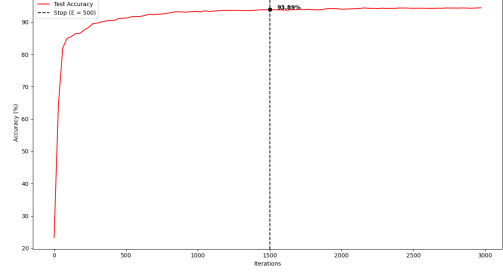


Figure 4: Accuracy vs Iterations ($\varepsilon = 500$) (Randomized BCGD)

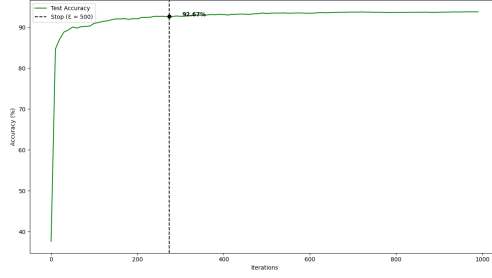


Figure 5: Accuracy vs Iterations ($\varepsilon = 500$) (Gauss-Southwell BCGD)

Table 1 shows the accuracy values achieved when using two different tolerances for the stopping condition.

Tolerance	Gradient Descent	Gauss-Southwell BCGD	Randomized BCGD
100	Iterations: 515 CPU Time: 1.25 s Accuracy: 94.47%	Iterations: 2537 CPU Time: 3.33 s Accuracy: 94.37%	Iterations: 11,500 CPU Time: 4.36 s Accuracy: 94.84%
500	Iterations: 90 CPU Time: 0.20 s Accuracy: 94.06%	Iterations: 274 CPU Time: 0.41 s Accuracy: 92.67%	Iterations: 1500 CPU Time: 0.57 s Accuracy: 93.89%

Table 1: Comparison of Optimization Algorithms by Stopping Tolerance

5.3 Hyperparameter Dynamics: Stepsize and Tolerance

This section discusses how the choice of the stepsize length and the stopping condition tolerance influences the performance of the implemented algorithms.

Stepsize Scale (L_{tot} vs L_j): As discussed in Section 3.7, from a theoretical perspective, convergence is guaranteed when stepsizes are set to $1/L$ and $1/L_j$ for GD and BCGD algorithms, respectively. However, since these represent conservative stepsizes designed for the worst-case scenario, introducing a stepsize multiplier h and computing updates as h/L and h/L_j is computationally more efficient.

During the hyperparameter tuning phase, with a fixed tolerance of 500, it was observed that multipliers of $h = 50$ for GD, $h = 5$ for Randomized BCGD, and $h = 9$ for Gauss-Southwell BCGD provided the optimal trade-off between test accuracy and CPU time.

The significant gap between these multipliers derives from the structural differences in the algorithms. First, GD and the BCGD variants operate on different scales: L is computed on the entire matrix A , whereas L_j depends only on the j -th column of A . Since $L \geq \max(L_j)$, the stepsizes have different magnitudes, explaining the considerable difference between 50 and 5 or 9.

Furthermore, although Randomized and Gauss-Southwell BCGD share the exact same L_j , the Gauss-Southwell rule allows for a larger multiplier ($h = 9$) because it explicitly updates in the direction of steepest coordinate descent. In contrast, the random selection in Randomized BCGD requires a more conservative multiplier ($h = 5$) to avoid the risk of overshooting the minimum.

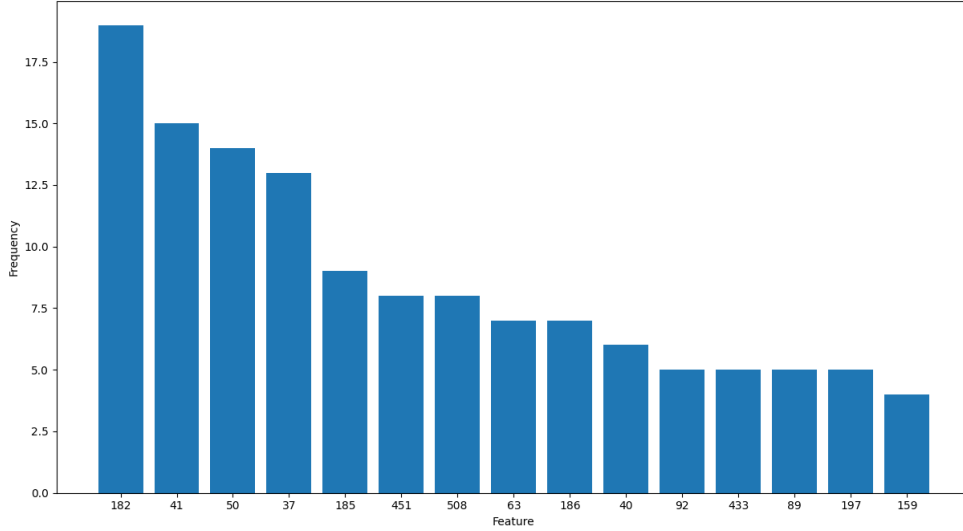


Figure 6: Feature update frequency in Gauss-Southwell BCGD.

Tolerance Sensitivity (Accuracy vs Tolerance): Although a tolerance of 500 was fixed to guarantee a fair comparison between the three algorithms, the effect of decreasing the threshold from 500 down to 100 was also investigated (Figure 7). As observed, standard Gradient Descent (GD) stabilizes rapidly: its accuracy only increases from 94.06% at $\varepsilon = 500$ to 94.47% at $\varepsilon = 100$, suggesting that for full-gradient methods, increasing the strength of the stopping condition produces marginal effects once a stable descent direction is set.

In contrast, the two BCGD variants show a steeper accuracy curve. Gauss-Southwell BCGD, in particular, is characterized by the most significant sensitivity: it starts at the lowest accuracy of the three by a margin (92.67% at $\varepsilon = 500$), but quickly recovers as the tolerance decreases, reaching a 94.37% accuracy at $\varepsilon = 100$. This behavior is caused by its greedy nature: at high tolerances, GS focuses exclusively on the most relevant features (e.g., Feature 182 in Figure 6), leaving others intact. However, when the tolerance is

lowered, the algorithm is forced to update less relevant features, closing the performance gap.

Randomized BCGD follows a similar trend, starting at 93.89% at $\varepsilon = 500$ and eventually reaching the highest overall accuracy (94.84%) at $\varepsilon = 100$, even surpassing the GD curve. This suggests that single coordinate updates allow for a more accurate weight approximation than the full-gradient approach when the tolerance threshold is low enough.

If the tolerance was set to $\varepsilon \approx 0$ to allow for full convergence, all three methods would eventually reach a similar accuracy of $\approx 94.8\% - 94.9\%$, effectively minimizing the differences between the algorithms.

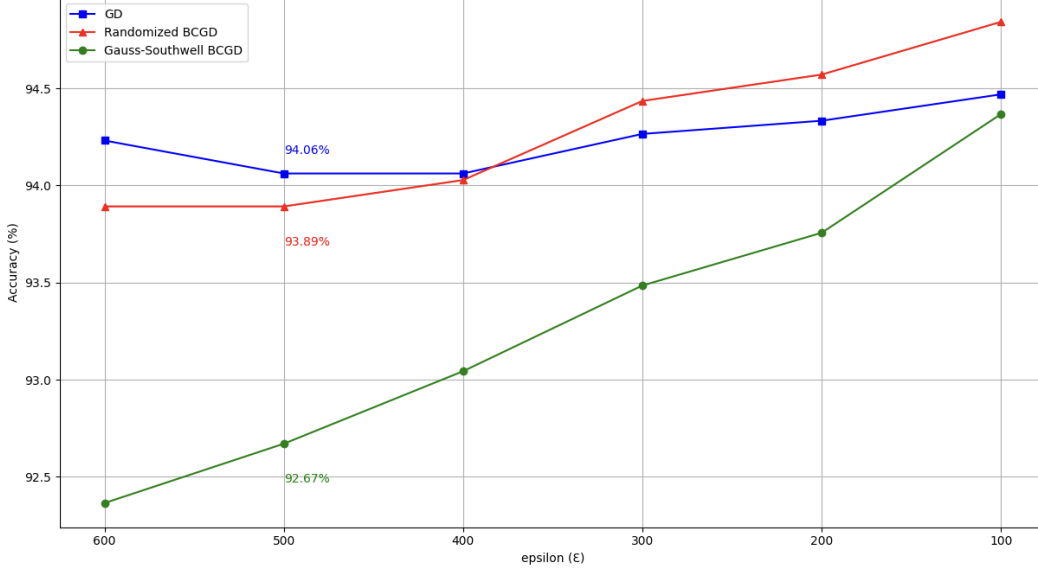


Figure 7: Accuracy of the algorithms vs tolerance (fixed stepsizes).

5.4 Decision Boundaries and Error Analysis

This final section presents a qualitative analysis of the models' ability to correctly separate the activity classes. This is achieved through a 2D projection of the feature space and a targeted visual representation of the prediction errors.

Due to the high dimensionality of the data matrix A , which contains 561 features (see Section 4), the data is projected onto a 2-dimensional space using Principal Component Analysis (PCA), as shown in Figure 8. This representation allows for a visualization of the spatial distribution of the different activities, even though compressed into only two principal components.

Targeted Error Analysis: From the analysis of the scatter plots in Figure 9, 10, 11, a common pattern of misclassified points emerges across all three algorithms. The prediction errors (highlighted in red) are concentrated in two specific areas:

- **The Dynamic Cluster:** Most of the errors are found in the dynamic region (*Walking*, *Walking Upstairs*, and *Walking Downstairs*). Collected data during dynamic activities can be noisy due to complex body movements, and accelerometer patterns for walking on a flat surface versus going up or down the stairs present significant overlapping features.

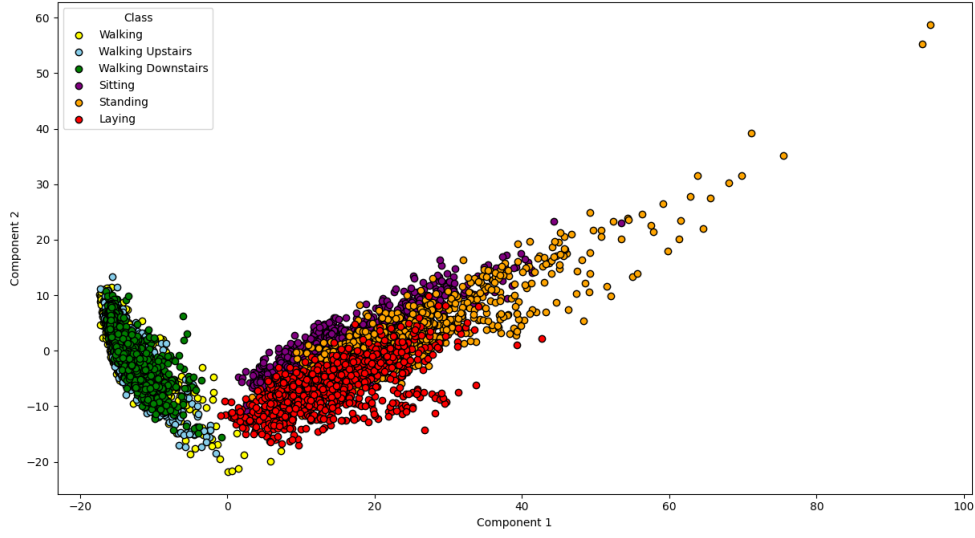


Figure 8: Explorative PCA projection of the dataset.

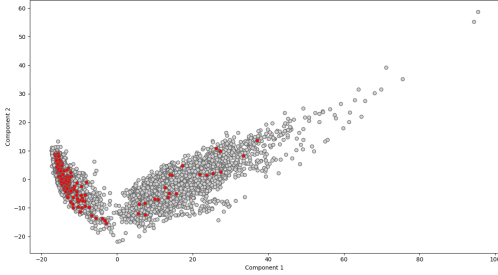


Figure 9: Prediction errors on the train set (GD)

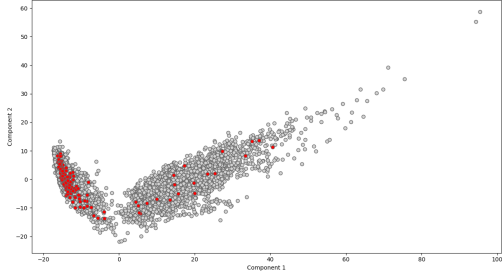


Figure 10: Prediction errors on the train set (Randomized BCGD)

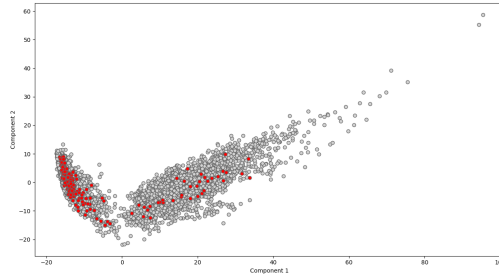


Figure 11: Prediction errors on the train set (Gauss-Southwell BCGD)

- **The Static Cluster:** The remaining errors occur between static activities, primarily *Sitting* and *Standing*. Both these activities are characterized by an absence of dynamic movement, producing similar feature detection that could lead to misclassification.

Overall, the fact that errors are localized in ambiguous regions, and not spread randomly, confirms that the models have successfully learned the patterns without overfitting the training data.

6 LLM Usage Disclosure Statement

We declare the use of LLMs only for individual study support purposes. We utilized AI tools (Gemini 3.1 between April 5th and May 3rd, 2026), in particular to:

- Debug the code and consult library documentation such as Scikit-learn, NumPy and Matplotlib to support the writing of the code related to the plots in the report
- Improve the general grammar and syntax of the text
- Learn more about LaTeX syntax for formulas and document formatting

References

- [1] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. Human activity recognition using smartphones. UCI Machine Learning Repository, 2012. Available at Kaggle: <https://www.kaggle.com/datasets/uciml/human-activity-recognition-with-smartphones>.